

Dynamic Scene Recreation Pipeline for Planning Environments

Albaraa Ammar Othman^{1,2}, Prabhneet Singh^{1,2}, Emanuele De Pellegrin², Maria Koskinopoulou³,
Ronald P. A. Petrick¹

¹Department of Computer Science, Heriot-Watt University, Edinburgh, Scotland, UK

²School of Informatics, University of Edinburgh, Edinburgh, Scotland, UK

³School of Engineering and Physical Sciences, Heriot-Watt University, Edinburgh, Scotland, UK

ao2000@hw.ac.uk, ps115@hw.ac.uk, edepell@ed.ac.uk, M.Koskinopoulou@hw.ac.uk, R.Petrick@hw.ac.uk

Abstract

Dynamic Scene Reconstruction is the complex task of creating a virtual counterpart to a real environment. However, recent advancements in foundation models have demonstrated their emerging ability to interpret elements and properties in scenes. While the use of these models in real-world domains remains an open challenge, coupled with sensor data, they provide a useful tool for generating the semantics of a scene. This demonstration presents a new framework that uses depth cameras and recent foundation models to identify and isolate objects within a scene and encode their properties into a symbolic representation in PDDL. This representation is suitable for plan generation, execution, and visualisation, with scene elements reconstructed in the Planning Domain Simulation (PDSim) environment.

Introduction

Dynamic Scene Reconstruction is the task of creating a virtual counterpart of a real physical environment. While such environments can be used for a range of diverse tasks involving autonomous agents, including training, teleoperation, navigation, decision making, or monitoring as a digital twin (Bavelos et al. 2025; Gu et al. 2023; Ni et al. 2025), reconstructing an environment in a virtual form and understanding its dynamics and properties is a challenging task.

While the problem of identifying objects and object properties has a long history in areas like vision, sensing, and robotics, with both manual and automated approaches being proposed (Redmon et al. 2016; Chen et al. 2019; Schneider et al. 2009; Vidal et al. 2023), recent advances in Generative AI provide new tools to aid in this process. Moreover, since visualization tools and game engines are now commonly used for building simulation systems, the need for approaches that facilitate the process of reconstructing scenes for such systems is seen as an important area of research.

In this demonstration, we consider the problem of automatically identifying objects and their spatial properties in order to build a symbolic representation that supports automated planning and plan visualisation. A combination of depth cameras and foundation models are used to induce a set of objects and object properties. This representation is



Figure 1: Example environment.

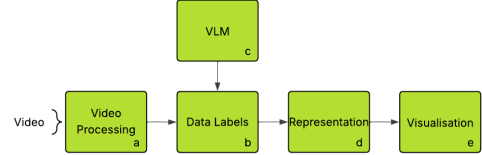


Figure 2: System Diagram

then converted into a PDDL domain and problem (McDermott et al. 1998), capable of being used for classical planning, providing a pipeline for converting continuous visual data into discrete symbolic data supporting planning model concepts. Given the induced planning model, we reconstruct a visualisation of the scene using PDSim (De Pellegrin and Petrick 2024), an open-source planning domain simulation engine built within the Unity Game Engine (Unity Technologies 2025). This visualisation enables plans to be generated and executed in the virtual environment.

Scene Reconstruction Pipeline

To form a symbolic representation of a discrete continuous space and then build our simulated domain, we use the pipeline in Figure 2. To support our demonstration, we mimic a manufacturing environment with LEGO Bricks (Figure 1, left). When stacking LEGO bricks the relative position of each brick matters which increases the complexity of the domain. To simplify our model we treat stacks as simply one brick on top of another equivalent to the Blocks World representation.

Predicate	Description
(Brick ?b)	?b is a brick
(Location ?l)	?l is a location
(Dimension ?d)	?d is a dimension.
(Colour ?c)	?c is an RGB colour
(Label ?la)	?la is a text label.
(brickAt ?b ?l)	Brick ?b is at location ?l
(on ?b ?base)	Brick ?b is on brick ?base
(clear ?b)	No brick is on brick ?b
(brickDimension ?b ?d)	Brick ?b scaled to dimension ?d.
(brickColour ?b ?c)	Brick ?b is colour ?c
(brickLabel ?b ?la)	Brick ?b has label ?la

Table 1: PDDL scene predicates.

Data Capture

A RealSense D435 depth camera is used to capture RGB and depth videos at a recording resolution of 640x480 pixels at 15 frames per second over a 30 second recording window with the camera positioned perpendicular to the table. We capture this depth data to provide a realistic visual representation in the later stages.

Video Processing

The pipeline begins (Figure 2a) by extracting the colour and depth frames from the recordings. Individual LEGO pieces are then discovered within the table region using SAM2’s Automatic Mask Generator (Ravi et al. 2024), tracked across keyframes, and de-duplicated. Finally, each piece’s mask pixels are projected into 3D camera space using the pinhole camera model and aligned depth data, giving physical dimensions (width, length, height in mm) and a relative XY position on the table.

Data Labelling This stage (Figure 2b) uses an external labelling tool to assign descriptor labels to each identified element in the scene. For this demonstration we use Vision Language Models (VLMs), specifically the qwen3-vl:235b-cloud model through Ollama. The VLM is provided with each identified mask and the prompt: Label this LEGO element using a maximum of 3 words. Do not label the colour. The first three words from each response form the item description. Often the VLM included the colour in the generated description which led to the constrained prompt.

Representation This stage (Figure 2d) converts the VLM-induced labels into a partial automated planning representation. Objects and object properties are converted into their PDDL equivalents and converted to Python for use with the Unified Planning Library (UPL) (Micheli et al. 2025). We consider each element of a planning problem $\Pi = \langle P, A, I, G \rangle$. Properties (P) are manually defined as seen in Table 1. Actions (A) are prespecified mimicking a typical Blocks World domain which includes actions for moving blocks and placing or removing blocks from a stack. The initial state (I) is automatically built using outputs from the VLM. Each brick ?b is created and assigned to a Cartesian

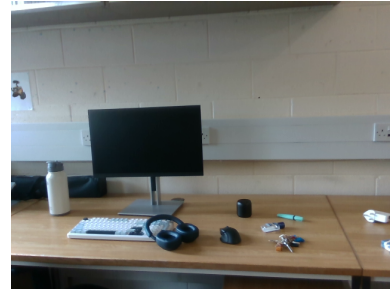


Figure 3: Office Space Example

location ?l, an RGB colour ?c, 3D dimensions ?d and a label ?la. As our task is to recreate the scene, the goal (G) for this demo is the simple stacking of all objects.

Visualisation At this stage of the pipeline (Figure 2e), the Planning Domain Simulation (PDSim) system (Pellegrin and Petrick 2024) is used to visualise the planning domain and automatically build our domain within Unity. All rendered objects are initially basic cubes at the same coordinate location. To visualise movements and arrange the scene, we manually create animations for predicates using Unity Script Graphs. In these graphs each node is an operation within Unity. When a plan is executed, the Script Graph for each predicate is automatically triggered upon reference and the animation is run. Nodes in these graphs can be customised using the Unity or PDSim libraries, which include nodes for translating objects to the positions of other objects. After the creation of the Script Graphs the domain will be initialised (Figure 1, right).

Conclusion

Following this process the plans can be executed and visualised within Unity. For instance, reaching the goal mentioned where all LEGO elements are stacked. Although we test on LEGO related scenes this pipeline is not constrained to this task. This pipeline can be taken and applied to a majority of scenes and where necessary aspects of the pipeline can be switched for more task specific software.

The demonstration video found here on Youtube - <https://youtu.be/2-7EH5sT0rk> - will step through the entire pipeline using the same LEGO example and also apply the pipeline to the office space scene found in Figure 3.

References

- Bavelos, A. C.; Anastasiou, E.; Dimitropoulos, N.; Michalos, G.; and Makris, S. 2025. Virtual reality-based dynamic scene recreation and robot teleoperation for hazardous environments. *Computer-Aided Civil and Infrastructure Engineering*, 40(3): 392–408.
- Chen, K.; Wang, J.; Pang, J.; Cao, Y.; Xiong, Y.; Li, X.; Sun, S.; Feng, W.; Liu, Z.; Xu, J.; Zhang, Z.; Cheng, D.; Zhu, C.; Cheng, T.; Zhao, Q.; Li, B.; Lu, X.; Zhu, R.; Wu, Y.; Dai, J.; Wang, J.; Shi, J.; Ouyang, W.; Loy, C. C.; and Lin, D. 2019. MMDetection: Open MMLab Detection Toolbox and Benchmark. arXiv:1906.07155.

De Pellegrin, E.; and Petrick, R. P. 2024. Planning domain simulation: an interactive system for plan visualisation. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 34, 133–141.

Gu, Q.; Kuwajerwala, A.; Morin, S.; Jatavallabhula, K. M.; Sen, B.; Agarwal, A.; Rivera, C.; Paul, W.; Ellis, K.; Chellappa, R.; Gan, C.; de Melo, C. M.; Tenenbaum, J. B.; Torralba, A.; Shkurti, F.; and Paull, L. 2023. ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning. *arXiv:2309.16650*.

McDermott, D.; Ghallab, M.; Howe, A.; Knoblock, C.; Ram, A.; Veloso, M.; Weld, D.; and Wilkins, D. 1998. PDDL—The planning domain definition language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control.

Micheli, A.; Bit-Monnot, A.; Röger, G.; Scala, E.; Valentini, A.; Framba, L.; Rovetta, A.; Trapasso, A.; Bonassi, L.; Gerevini, A. E.; et al. 2025. Unified Planning: Modeling, manipulating and solving AI planning problems in Python. *SoftwareX*, 29: 102012.

Ni, Z.; Hupkes, J.; Eriksson, P.; Leijonhufvud, G.; Karlsson, M.; and Gong, S. 2025. Parametric Digital Twins for Preserving Historic Buildings: A Case Study at Löfstad Castle in Östergötland, Sweden. *IEEE Access*, 13: 3371–3389.

Pellegrin, E. D.; and Petrick, R. 2024. Simulating Robotics Planning Domains with PDSim and ROS. In *ICAPS 2024 System's Demonstration track*.

Ravi, N.; Gabeur, V.; Hu, Y.-T.; Hu, R.; Ryali, C.; Ma, T.; Khedr, H.; Rädle, R.; Rolland, C.; Gustafson, L.; Mintun, E.; Pan, J.; Alwala, K. V.; Carion, N.; Wu, C.-Y.; Girshick, R.; Dollár, P.; and Feichtenhofer, C. 2024. SAM 2: Segment Anything in Images and Videos. *arXiv preprint arXiv:2408.00714*.

Redmon, J.; Divvala, S.; Girshick, R.; and Farhadi, A. 2016. You Only Look Once: Unified, Real-Time Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Schneider, A.; Sturm, J.; Stachniss, C.; Reiser, M.; Burkhardt, H.; and Burgard, W. 2009. Object identification with tactile sensors using bag-of-features. In *2009 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 243–248.

Unity Technologies. 2025. Unity. Game development platform.

Vidal, J.; Vallicrosa, G.; Martí, R.; and Barnada, M. 2023. Brickognize: Applying Photo-Realistic Image Synthesis for Lego Bricks Recognition with Limited Data. *Sensors*, 23(4): 1898.